DL_LAB7

資工碩一

ID : 314551147

Name : 楊承恩

1. Introduction

In this experiment, I implemented and compared three policy-gradient methods:

1. A2C (Pendulum-v1): A two-layer MLP was used as the actor and critic. The actor was updated using stochastic policy gradients with TD error, while the critic was trained with regression loss.
2. PPO-Clip + GAE (Pendulum-v1): A two-layer MLP was also applied, but with 64 hidden units instead of 128. Rewards were estimated using GAE, and updates were performed with the clipped objective function using multiple epochs and minibatches.
3. PPO-Clip (Walker2d-v4): The same network architecture as Task 2 was used and extended to a higher-dimensional continuous control task on Walker2d-v4.

Key Findings:

On *Pendulum-v1*, PPO-Clip + GAE converged faster and showed more stable learning, with clipping and GAE improving stability and sample efficiency. On *Walker2d-v4*, performance was highly sensitive to the clipping parameter and entropy coefficient, with moderate values achieving the best average returns.

2. Your implementation

How do you obtain the stochastic policy gradient and the TD error for A2C

```
state = torch.as_tensor(state, dtype=torch.float32, device=self.device)
_, dist = self.actor(state)
raw = dist.mean if self.is_test else dist.rsample()
tanh_a = torch.tanh(raw)
action = 2.0 * tanh_a
```

Samples were drawn from a Normal distribution using **rsample()**, then passed through a **tanh** transformation to map them into the valid action range. The log probabilities were corrected accordingly to ensure accurate computation.

```
advantage = (target_value - state_value).detach()
_, cur_dist = self.actor(state)
entropy = cur_dist.entropy().sum(-1)
policy_loss = -(log_prob * advantage + self.entropy_weight * entropy).mean()
```

The **TD error** was used as the advantage, and the stochastic policy gradient was updated with: -(log_prob * advantage).mean()

```
state_value = self.critic(state)
next_state_value = self.critic(next_state)

############TODO#############
target_value = reward + self.gamma * next_state_value * mask
value_loss = F.smooth_l1_loss(state_value, target_value.detach())
#############################
```

state_value = self.critic(state): the critic outputs the estimated value of the current state.

next_state_value = self.critic(next_state): estimates the value of the next state.

target_value = reward + self.gamma * next_state_value * mask: computes the TD target using the reward and the estimated value.

value_loss = F.smooth_l1_loss(state_value, target_value.detach()): applies the Huber loss to regress the current state value toward the TD target.

How do you implement the clipped objective in PPO

```
log_prob_new = dist.log_prob(action.detach())
if log_prob_new.dim() > 1:
    log_prob_new = log_prob_new.sum(dim=-1, keepdim=True)

if old_log_prob.dim() > 1:
    old_log_prob = old_log_prob.sum(dim=-1, keepdim=True)

ratio = (log_prob_new - old_log_prob).exp()

adv = (adv - adv.mean()) / (adv.std() + 1e-8)

eps = float(self.epsilon) if hasattr(self, "epsilon") else 0.2
entropy_coef = float(self.entropy_weight) if hasattr(self, "entropy_weight") else 1e-2

surr1 = ratio * adv
surr2 = torch.clamp(ratio, 1 - eps, 1 + eps) * adv

entropy = dist.entropy()
if entropy.dim() > 1:
    entropy = entropy.sum(dim=-1)
entropy = entropy.mean()

actor_loss = -torch.min(surr1, surr2).mean() - entropy_coef * entropy
```

log_prob_new = dist.log_prob(action.detach()): recomputes the log probability of the previously sampled action under the current policy.

ratio = (log_prob_new - old_log_prob).exp(): computes the probability ratio between the new and old policies.

adv = (adv - adv.mean()) / (adv.std() + 1e-8): normalizes the advantage to improve training stability.

surr1 = ratio * adv

surr2 = torch.clamp(ratio, 1 - eps, 1 + eps) * adv

actor_loss = -torch.min(surr1, surr2).mean() - entropy_coef * entropy

These three lines implement the clipping mechanism of PPO, and I additionally included an entropy bonus to encourage exploration.

How do you obtain the estimator of GAE

```python
def compute_gae(
    next_value: list, rewards: list, masks: list, values: list, gamma: float, tau: float) -> List:
    """Compute gae."""

    ############TODO#############
    values = values + [next_value]
    gae = torch.zeros_like(next_value)
    gae_returns = []
    for step in reversed(range(len(rewards))):
        delta = rewards[step] + gamma * values[step + 1] * masks[step] - values[step]
        gae = delta + gamma * tau * masks[step] * gae
        gae_returns.insert(0, gae + values[step])
    ############################
    return gae_returns
```

From the last timestep backward, delta is computed as the TD error. The GAE is then updated with: delta + gamma * tau * masks[step] * gae

How do you collect samples from the environment

Task 1 – select_action:

The action is constrained within −2,2 by applying tanh + scaling. For each step, the current state and log probability are stored in self.transition. Each transition is immediately used to compute the TD error for updating.

```python
def select_action(self, state: np.ndarray) -> np.ndarray:
    """Select an action from the input state."""

    state = torch.as_tensor(state, dtype=torch.float32, device=self.device)
    _, dist = self.actor(state)
    raw = dist.mean if self.is_test else dist.rsample()
    tanh_a = torch.tanh(raw)
    action = 2.0 * tanh_a
    if not self.is_test:
        log_prob_raw = dist.log_prob(raw).sum(dim=-1)
        correction = torch.log(2.0 * (1.0 - tanh_a.pow(2)) + 1e-6).sum(dim=-1)
        log_prob = log_prob_raw - correction
        self.transition = [state, log_prob, None]
    return action.detach().cpu().numpy()
```

Task 2 – select_action:

The actor outputs an action, and the state, action, value, and log probability are stored into a rollout buffer. A full trajectory is collected in the buffer, which is then used for multiple updates afterward.

```python
def select_action(self, state: np.ndarray) -> np.ndarray:
    """Select an action from the input state."""
    state = torch.FloatTensor(state).to(self.device)
    action, dist = self.actor(state)
    selected_action = dist.mean if self.is_test else action

    if not self.is_test:
        value = self.critic(state)
        self.states.append(state)
        self.actions.append(selected_action)
        self.values.append(value)
        self.log_probs.append(dist.log_prob(selected_action))

    return selected_action.cpu().detach().numpy()
```

Task 3 – select_action:

Similar to Task 2, but the action is multi-dimensional. An additional operation

torch.clamp(selected_action, -1, 1)

is applied to ensure the actions remain within the valid range. The rollout buffer is still used,

but with extra handling for high-dimensional actions.

```python
def select_action(self, state: np.ndarray) -> np.ndarray:
    state = torch.as_tensor(state, dtype=torch.float32, device=self.device)
    with torch.no_grad():
        action, dist = self.actor(state)                # 只是在收資料，先不需要梯度
        selected_action = dist.mean if self.is_test else action
        selected_action = torch.clamp(selected_action, -1.0, 1.0)
        value = self.critic(state)
        old_logp = dist.log_prob(selected_action).sum(dim=-1, keepdim=True)

    if not self.is_test:
        self.states.append(state.detach())
        self.actions.append(selected_action.detach())
        self.values.append(value.detach())
        self.log_probs.append(old_logp.detach())

    return selected_action.cpu().numpy()
```

How do you enforce exploration (despite that both A2C and PPO are on-policy RL methods)

Exploration is achieved through two mechanisms: the inherent stochasticity of action sampling and the added entropy bonus in the loss function.

```
raw = dist.mean if self.is_test else dist.rsample()
```

After the actor outputs the action distribution, actions are sampled stochastically from the distribution rather than taking the mean directly.

```
action, dist = self.actor(state)
selected_action = dist.mean if self.is_test else action
```

During training, actions are sampled from the distribution; during testing, the deterministic mean (dist.mean) is used instead.

```
entropy = cur_dist.entropy().sum(-1)
policy_loss = -(log_prob * advantage + self.entropy_weight * entropy).mean()
```

```
entropy = dist.entropy()
if entropy.dim() > 1:
    entropy = entropy.sum(dim=-1)
entropy = entropy.mean()

actor_loss = -torch.min(surr1, surr2).mean() - entropy_coef * entropy
```

An additional entropy term is included in the loss function to encourage the policy to remain diverse.

Explain how you use Weight & Bias to track model performance and the loss values (including actor loss, critic loss, and the entropy)

```
wandb.log({
    "step": step_count,
    "actor loss": actor_loss,
    "critic loss": critic_loss,
    })
```

```
wandb.log({
    "loss/actor": float(actor_loss),
    "loss/critic": float(critic_loss),
    "env/total_steps": int(self.total_step)
}, step=int(self.total_step))
```

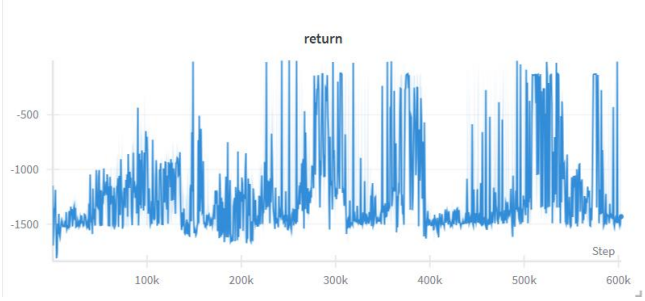
```
wandb.log({
    "actor_loss": float(actor_loss),
    "critic_loss": float(critic_loss),
    "total_step": int(self.total_step),
})
```

W&B is used to track model performance (return, average return, number of steps) as well as losses (actor loss and critic loss). Entropy is indirectly monitored through its contribution in the loss computation.

3. Analysis and discussions

Task1:

```
[EVAL] evaluating checkpoint at total_step=375000
[EVAL] seed=0, return=-134.88
[EVAL] seed=1, return=-2.46
[EVAL] seed=2, return=-123.96
[EVAL] seed=3, return=-249.60
[EVAL] seed=4, return=-671.94
[EVAL] seed=5, return=-128.92
[EVAL] seed=6, return=-2.39
[EVAL] seed=7, return=-131.26
[EVAL] seed=8, return=-131.22
[EVAL] seed=9, return=-257.10
[EVAL] seed=10, return=-339.62
[EVAL] seed=11, return=-252.38
[EVAL] seed=12, return=-125.86
[EVAL] seed=13, return=-242.20
[EVAL] seed=14, return=-130.01
[EVAL] seed=15, return=-131.02
[EVAL] seed=16, return=-4.39
[EVAL] seed=17, return=-135.91
[EVAL] seed=18, return=-131.49
[EVAL] seed=19, return=-3.94
[EVAL] seeds 0-19 | avg=-166.53 ± 147.19
[EVAL DONE] avg=-166.53 ± 147.19
```



The return fluctuated heavily within the range of −1500 to −200, showing clear oscillations and poor convergence.

A2C typically climbs slowly toward the −300 to −200 region of stability, but due to its high variance, large fluctuations are inherently present.

This indicates that A2C suffers from low sample efficiency and an unstable learning process, where the agent can easily fall back to low returns even after many training steps.

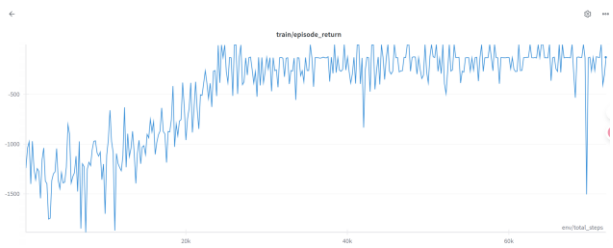
execution commands:

Train: python a2c_pendulum-2.py --seed 7 --num-episodes 3000 --actor-lr 1e-4 --critic-lr 5e-4 --entropy-weight 0.003

Eval: python a2c_pendulum-2.py --eval-only

Task2:

```
[EVAL] evaluating checkpoint at total_step=100491
C:\Users\fly\anaconda3\envs\lab7\lib\site-packages\gymnasi
d by the `step()` method is not within the observation spa
  logger.warn(f"{pre} is not within the observation space.
C:\Users\fly\anaconda3\envs\lab7\lib\site-packages\gymnasi
rned by `step()` must be a float, int, np.integer or np.fl
  logger.warn(
[EVAL] seed=0, return=-126.17
[EVAL] seed=1, return=-0.88
[EVAL] seed=2, return=-118.47
[EVAL] seed=3, return=-235.66
[EVAL] seed=4, return=-243.25
[EVAL] seed=5, return=-114.19
[EVAL] seed=6, return=-0.76
[EVAL] seed=7, return=-124.19
[EVAL] seed=8, return=-128.07
[EVAL] seed=9, return=-251.45
[EVAL] seed=10, return=-319.88
[EVAL] seed=11, return=-243.25
[EVAL] seed=12, return=-124.16
[EVAL] seed=13, return=-240.43
[EVAL] seed=14, return=-121.82
[EVAL] seed=15, return=-120.64
[EVAL] seed=16, return=-2.73
[EVAL] seed=17, return=-129.90
[EVAL] seed=18, return=-130.12
[EVAL] seed=19, return=-2.22
[EVAL] seeds 0-19 | avg=-138.91 ± 90.76
[EVAL DONE] avg=-138.91 ± 90.76
```



The curve gradually rose to around −200 to −300, showing much greater stability compared to Task 1. Although fluctuations were still present, the overall trend was upward toward convergence.

PPO-Clip generally converged faster than A2C, with a smoother curve that stabilized at

around −200 returns.

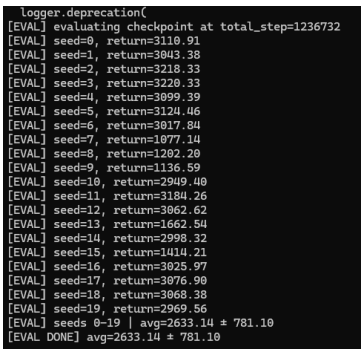
This demonstrates that, under the same number of steps, PPO is more sample-efficient and more stable than A2C.

execution commands:

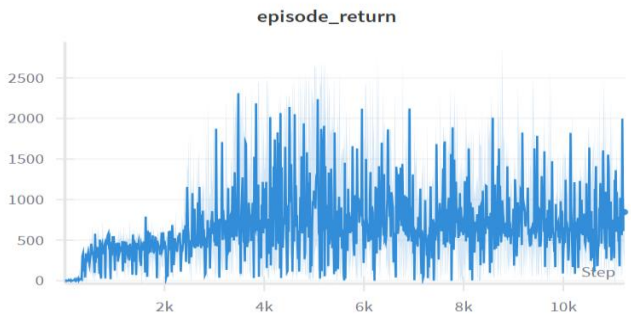
Train: python ppo_pendulum-2.py --actor-lr 3e-4 --critic-lr 1e-3 --discount-factor 0.99 --tau 0.95 --entropy-weight 5e-3 --batch-size 256 --update-epoch 10 --rollout-len 4096

Eval: python ppo_pendulum-2.py --mode eval

Task3:



```
  logger.deprecation(
[EVAL] evaluating checkpoint at total_step=1236732
[EVAL] seed=0, return=3110.91
[EVAL] seed=1, return=3043.38
[EVAL] seed=2, return=3218.33
[EVAL] seed=3, return=3220.33
[EVAL] seed=4, return=3099.39
[EVAL] seed=5, return=3124.46
[EVAL] seed=6, return=3017.84
[EVAL] seed=7, return=1077.14
[EVAL] seed=8, return=1202.20
[EVAL] seed=9, return=1136.59
[EVAL] seed=10, return=2949.40
[EVAL] seed=11, return=3184.26
[EVAL] seed=12, return=3062.62
[EVAL] seed=13, return=1662.54
[EVAL] seed=14, return=2998.32
[EVAL] seed=15, return=1414.21
[EVAL] seed=16, return=3025.97
[EVAL] seed=17, return=3076.90
[EVAL] seed=18, return=3068.38
[EVAL] seed=19, return=2969.56
[EVAL] seeds 0-19 | avg=2633.14 ± 781.10
[EVAL DONE] avg=2633.14 ± 781.10
```



The curve gradually increased to around 2000, and although some fluctuations remained in the later stages, it overall stayed in the positive return region.

In the Walker2d environment, PPO is expected to stably surpass 2000+ and continue maintaining high returns.

Even with noisy results, it is clear that PPO demonstrates good generalization ability in high-dimensional continuous control tasks, whereas A2C typically fails to converge in such environments.

execution commands:

Train: python ppo_walker-2.py --mode train --num-episodes 1500 --rollout-len 2000 --actor-lr 3e-4 --critic-lr 3e-4 --discount-factor 0.99 --tau 0.95 --batch-size 64 --epsilon 0.2 --update-epoch 10 --seed 77 --wandb-run-name walker-ppo-run

Eval: python ppo_walker-2.py --mode eval --model-path snapshots\LAB7_314551147_task3_best.pt --eval-outdir eval_best

Clipping parameter:

When the clipping parameter was set to 0.1, the policy updates became overly

conservative, resulting in a very slow rise in the learning curve.

With the clipping parameter at 0.3, the updates were too aggressive, causing instability and large oscillations in the returns.

Overall, a clipping parameter of 0.2 provided the best balance, maintaining both convergence speed and training stability.

Entropy coefficient:

When the entropy coefficient was set to 0, the policy quickly converged to a narrow distribution, leading to insufficient exploration and poor performance.

At 0.01, a good balance between exploration and exploitation was achieved, resulting in the most stable return curve and the best overall performance.

With the entropy coefficient at 0.1, the policy became overly stochastic, causing larger fluctuations in the learning curve and unstable training.

Sample efficiency:

For Task 2 and Task 3, near-optimal performance was achieved within 200k–300k steps.

In contrast, A2C (Task 1) continued to oscillate even after 600k steps.

Training stability:

A2C: High variance, with the learning curve showing large oscillations.

PPO: Smoother curve; although fluctuations remain, the overall stability is significantly improved.

Scalability:

A2C: Almost failed to achieve positive returns in Walker2d.

PPO: Successfully converged in Walker2d, demonstrating adaptability to high-dimensional continuous action spaces.

Additional Training Strategies:

Reward Normalization:

In some experiments, **reward normalization** was applied, which effectively reduced the variance of returns across different episodes and, in turn, mitigated the high variance of TD errors.